ECDSA 서명 시스템 분석

개념적 배경

왜 ECDSA가 필요한가?

Ethereum은 분산 네트워크에서 트랜잭션의 무결성과 인증을 보장하기 위해 디지털 서명 시스템이 필요합니다. ECDSA(Elliptic Curve Digital Signature Algorithm)는 다음과 같은 이유로 선택되었습니다:

1. **효율성**: RSA 대비 작은 키 크기로 동일한 보안 수준 제공
2. **성능**: 빠른 서명 생성 및 검증
3. **Bitcoin 호환성**: Bitcoin과 동일한 secp256k1 곡선 사용으로 상호 운용성 확보
4. **키 복구**: 서명에서 공개키를 복구할 수 있어 저장 공간 절약

수학적 원리

ECDSA는 타원곡선 암호학을 기반으로 하며, Ethereum은 secp256k1 곡선을 사용합니다:

```
y² = x³ + 7 (mod p)
```

여기서:

- $p = 2^{256} - 2^{32} - 2^{9} - 2^{8} - 2^{7} - 2^{6} - 2^{4} - 1$ (소수)
- 생성점 G = (Gx, Gy)
- 곡선의 차수 n = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141

구현 분석

핵심 데이터 구조

1. 타원곡선 정의 (`crypto/secp256k1/curve.go`)

```
type BitCurve struct {
P *big.Int // 유한체의 소수
N *big.Int // 기저점의 차수
B *big.Int // 곡선 방정식의 상수 (7)
Gx, Gy *big.Int // 기저점 좌표
BitSize  int  // 필드 크기 (256)
}
```

2. 서명 구조

Ethereum의 서명은 65바이트로 구성됩니다:

- R (32바이트): 서명의 첫 번째 구성요소
- S (32바이트): 서명의 두 번째 구성요소
- V (1바이트): 복구 ID (0 또는 1, 체인 ID에 따라 조정)

```
const  SignatureLength = 64 + 1  // 64 bytes ECDSA signature + 1 byte recovery i
const  RecoveryIDOffset = 64
const  DigestLength = 32
```

주요 알고리즘

1. 개인키 생성 및 검증

```go
func toECDSA(d []byte, strict bool) (*ecdsa.PrivateKey, error) {
priv := new(ecdsa.PrivateKey)
priv.PublicKey.Curve = S256()
// 키 길이 검증
if strict && 8*len(d) != priv.Params().BitSize {
return nil, fmt.Errorf("invalid length, need %d bits", priv.Params().BitSize)
}
priv.D = new(big.Int).SetBytes(d)
// 개인키는 0 < D < N 범위에 있어야 함
if priv.D.Cmp(secp256k1N) >= 0 {
return nil, errors.New("invalid private key, >=N")
}
if priv.D.Sign() <= 0 {
return nil, errors.New("invalid private key, zero or negative")
}
// 공개키 계산: Q = D * G
priv.PublicKey.X, priv.PublicKey.Y = S256().ScalarBaseMult(d)
if priv.PublicKey.X == nil {
return nil, errors.New("invalid private key")
}
return priv, nil
```

```
}
```

2. 서명 생성 (CGO 버전)

```
func Sign(digestHash []byte, prv *ecdsa.PrivateKey) (sig []byte, err error) {
if len(digestHash) != DigestLength {
return nil, fmt.Errorf("hash is required to be exactly %d bytes (%d)",
DigestLength, len(digestHash))
}
// 개인키를 바이트 배열로 변환
seckey := math.PaddedBigBytes(prv.D, prv.Params().BitSize/8)
defer zeroBytes(seckey) // 보안을 위한 메모리 정리
// libsecp256k1을 사용한 서명 생성
return secp256k1.Sign(digestHash, seckey)
}
```

3. 서명 검증

```
func VerifySignature(pubkey, digestHash, signature []byte) bool {
return secp256k1.VerifySignature(pubkey, digestHash, signature)
}
```

4. 공개키 복구

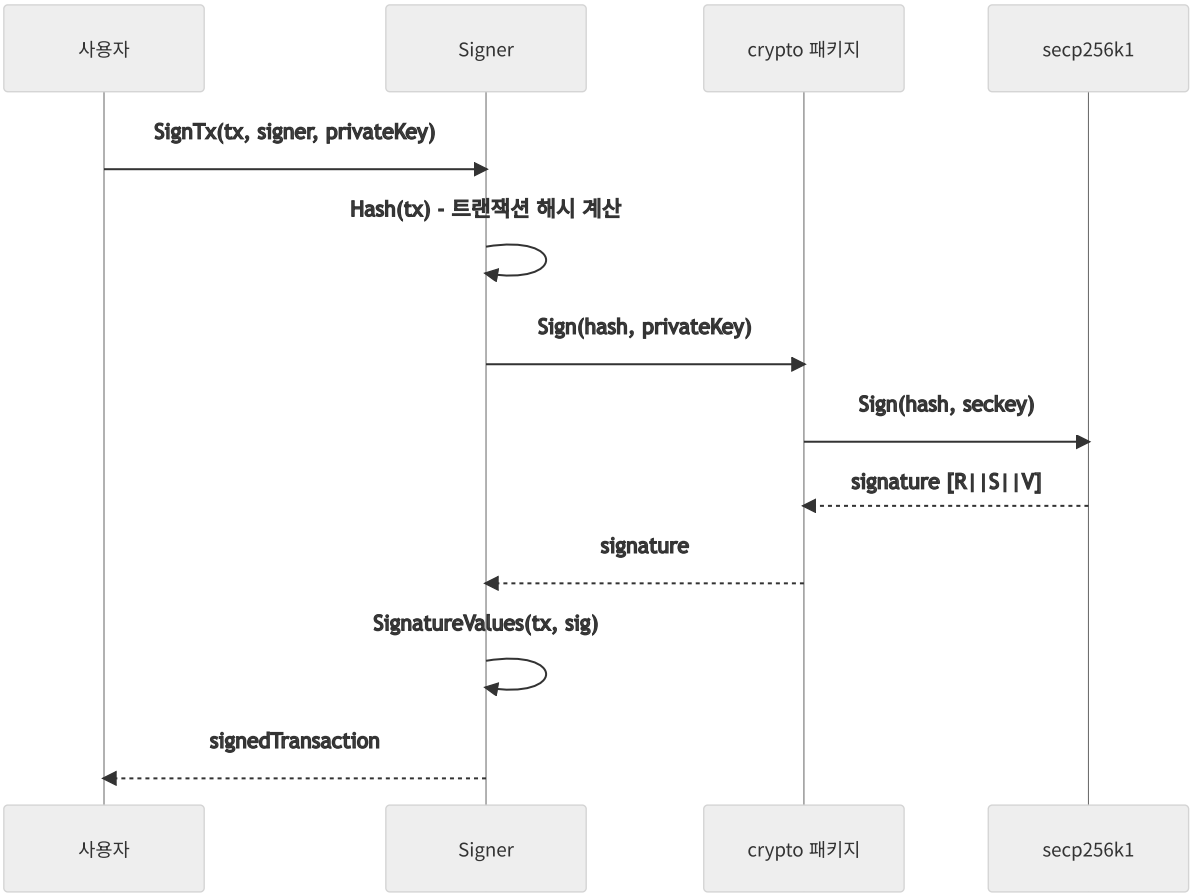
```go
func Ecrecover(hash, sig []byte) ([]byte, error) {
return secp256k1.RecoverPubkey(hash, sig)
}

func SigToPub(hash, sig []byte) (*ecdsa.PublicKey, error) {
s, err := Ecrecover(hash, sig)
if err != nil {
return nil, err
}
return UnmarshalPubkey(s)
}
```

코드 구조 및 모듈 관계

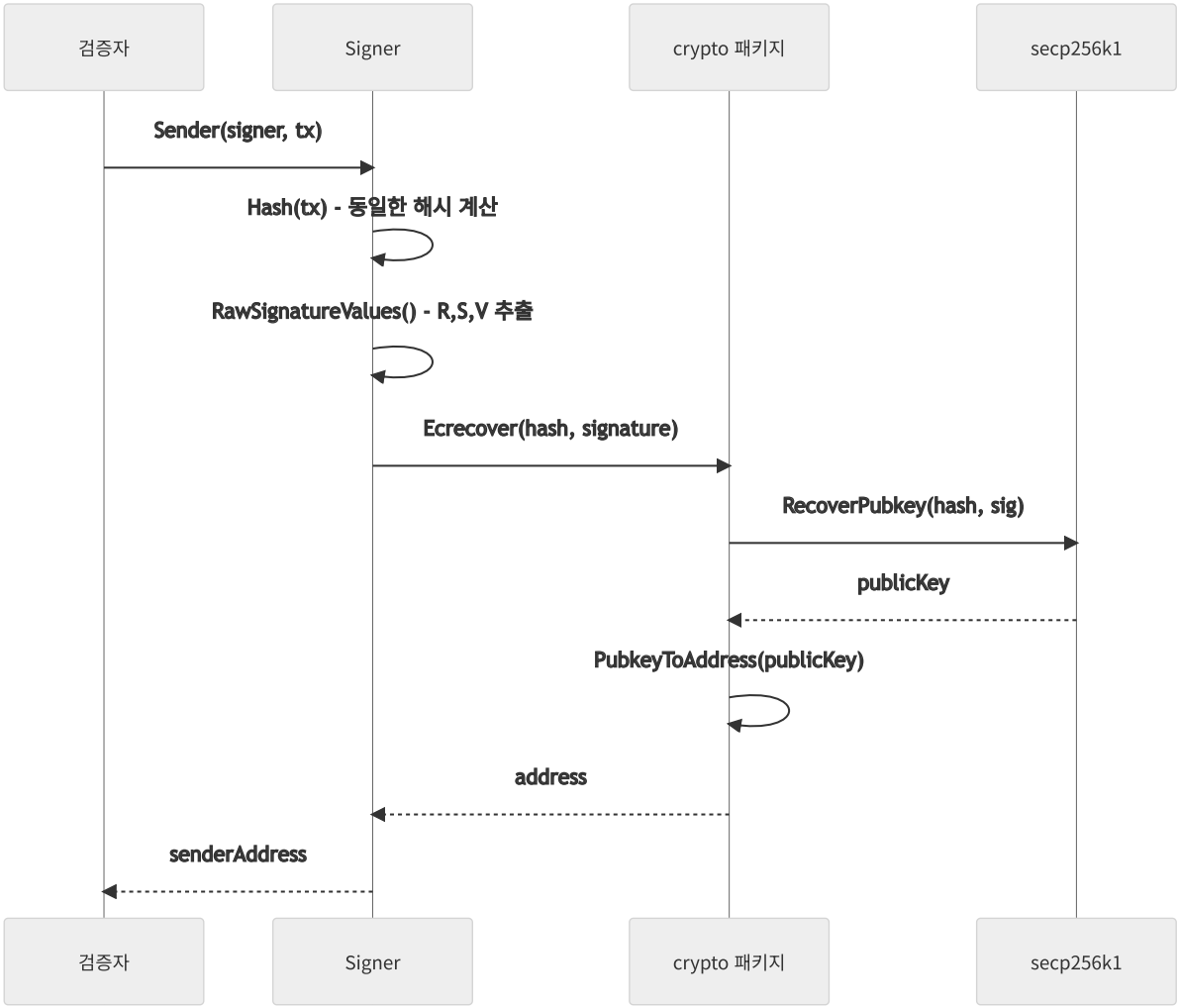
```
crypto/
├── crypto.go # 고수준 암호화 함수들
├── signature_cgo.go # CGO 기반 서명 구현
├── signature_nocgo.go # 순수 Go 서명 구현
└── secp256k1/
  ├── secp256.go # libsecp256k1 래퍼
  ├── curve.go # 타원곡선 구현
  └── libsecp256k1/ # C 라이브러리
```

실행 흐름

1. 트랜잭션 서명 과정



2. 서명 검증 과정



3. 단계별 실행 과정

서명 생성 단계:

1. **해시 계산**: 트랜잭션 데이터를 RLP 인코딩 후 Keccak256 해시
2. **논스 생성**: RFC6979 결정적 논스 생성
3. **서명 계산**:

- k = 임시 개인키 (논스)
- r = (k * G).x mod n
- s = k^{-1}(hash + r * privateKey) mod n

4. **복구 ID 계산**: 공개키 복구를 위한 추가 정보
5. **서명 인코딩**: [R || S || V] 형태로 65바이트 서명 생성

서명 검증 단계:

1. **서명 파싱**: 65바이트 서명을 R, S, V로 분리
2. **서명 유효성 검사**:

- 0 < r < n
- 0 < s < n
- s ≤ n/2 (malleability 방지)

3. **공개키 복구**:

- 타원곡선 점 복구
- 복구 ID를 사용한 올바른 점 선택

4. **주소 계산**: 공개키에서 Ethereum 주소 도출

성능 최적화

1. 이중 구현 전략

geth는 성능과 호환성을 위해 두 가지 구현을 제공합니다:

CGO 버전 (`signature_cgo.go`)

- **장점**: libsecp256k1 C 라이브러리 사용으로 최고 성능
- **단점**: CGO 의존성, 크로스 컴파일 복잡성

- **사용 조건**: `cgo` 빌드 태그가 활성화된 경우

순수 Go 버전 (`signature_nocgo.go`)

- **장점**: 순수 Go 구현으로 이식성 우수
- **단점**: C 버전 대비 성능 저하
- **사용 조건**: CGO 비활성화 또는 특정 플랫폼

2. 메모리 최적화

```
// 개인키 메모리 정리
seckey := math.PaddedBigBytes(prv.D, prv.Params().BitSize/8)
defer zeroBytes(seckey) // 메모리에서 개인키 제거

func zeroBytes(bytes []byte) {
clear(bytes) // Go 1.21+ 최적화된 메모리 클리어
}
```

3. 서명 캐싱

```
type sigCache struct {
signer Signer
from common.Address
}

func Sender(signer Signer, tx *Transaction) (common.Address, error) {
```

```
if sigCache := tx.from.Load(); sigCache != nil {
if sigCache.signer.Equal(signer) {
return sigCache.from, nil // 캐시된 결과 반환
}
}
// 새로운 계산 수행
addr, err := signer.Sender(tx)
if err != nil {
return common.Address{}, err
}
tx.from.Store(&sigCache{signer: signer, from: addr})
return addr, nil
}
```

4. 해시 풀링

```
var hasherPool = sync.Pool{
New: func() any {
return sha3.NewLegacyKeccak256().(KeccakState)
},
}

func Keccak256Hash(data ...[]byte) (h common.Hash) {
```

```
d := hasherPool.Get().(KeccakState)
d.Reset()
for _, b := range data {
d.Write(b)
}
d.Read(h[:])
hasherPool.Put(d) // 해시 객체 재사용
return h
}
```

실제 사용 예시

1. 트랜잭션 서명

```
// 개인키 생성
privateKey, err := crypto.GenerateKey()
if err != nil {
log.Fatal(err)
}

// 트랜잭션 생성
tx := types.NewTransaction(
nonce,
```

```
to,
value,
gasLimit,
gasPrice,
data,
)

// 서명자 생성
signer := types.NewEIP155Signer(chainID)

// 트랜잭션 서명
signedTx, err := types.SignTx(tx, signer, privateKey)
if  err != nil {
log.Fatal(err)
}
```

2. 서명 검증

```
// 서명자 주소 복구
sender, err := types.Sender(signer, signedTx)
if  err != nil {
log.Fatal(err)
}
```

```
// 예상 주소와 비교
expectedAddr := crypto.PubkeyToAddress(privateKey.PublicKey)
if sender != expectedAddr {
log.Fatal("서명 검증 실패")
}
```

3. 키 관리

```
// 개인키를 파일에서 로드
privateKey, err := crypto.LoadECDSA("keyfile")
if err != nil {
log.Fatal(err)
}

// 개인키를 파일에 저장
err = crypto.SaveECDSA("keyfile", privateKey)
if err != nil {
log.Fatal(err)
}

// 16진수 문자열에서 개인키 파싱
privateKey, err = crypto.HexToECDSA("hexstring")
```

```
if err != nil {
log.Fatal(err)
}
```

에러 처리 메커니즘

1. 서명 유효성 검증

```
func ValidateSignatureValues(v byte, r, s *big.Int, homestead bool) bool {
if r.Cmp(common.Big1) < 0 || s.Cmp(common.Big1) < 0 {
return false
}
// ECDSA malleability 방지 (EIP-2)
if homestead && s.Cmp(secp256k1halfN) > 0 {
return false
}
// 범위 검증
return r.Cmp(secp256k1N) < 0 && s.Cmp(secp256k1N) < 0 && (v == 0 || v == 1)
}
```

2. 체인 ID 검증

```
func (s EIP155Signer) Sender(tx *Transaction) (common.Address, error) {
```

```
if  tx.ChainId().Cmp(s.chainId) != 0 {

return common.Address{}, fmt.Errorf("%w: have %d want %d",

ErrInvalidChainId, tx.ChainId(), s.chainId)

}

// 서명 검증 계속...

}
```

3. 공통 에러 타입

```
var (

ErrInvalidSig = errors.New("invalid transaction v, r, s values")

ErrUnexpectedProtection = errors.New("transaction type does not supported EIP-15

ErrInvalidChainId = errors.New("invalid chain id for signer")

)
```

관련 시스템

의존성 관계

1. **core/types**: 트랜잭션 타입 정의 및 서명 인터페이스
2. **accounts**: 키 관리 및 서명 서비스
3. **core/txpool**: 트랜잭션 검증 시 서명 확인
4. **eth/protocols**: P2P 통신 시 트랜잭션 전파

상호작용 패턴

- **트랜잭션 풀**: 새 트랜잭션 수신 시 서명 검증
- **블록 검증**: 블록 내 모든 트랜잭션 서명 확인
- **RPC API**: 사용자 요청 시 트랜잭션 서명 생성
- **마이닝**: 블록 생성 시 트랜잭션 서명 재검증

영향 범위

ECDSA 서명 시스템의 변경은 다음 영역에 영향을 미칩니다:

1. **호환성**: 다른 Ethereum 클라이언트와의 상호 운용성
2. **성능**: 트랜잭션 처리 속도 및 네트워크 처리량
3. **보안**: 전체 네트워크의 암호학적 보안 수준
4. **저장소**: 서명 데이터 크기 및 블록체인 크기

이 ECDSA 서명 시스템은 Ethereum의 보안과 무결성을 보장하는 핵심 구성요소로, 모든 트랜잭션과 블록의 인증에 사용됩니다.